

ЛАБОРАТОРНАЯ РАБОТА № 3

Архитектура памяти Windows

Ход выполнения лабораторной работы

1. Создать форму с элементами для вывода информации о занятой памяти;
2. Получить информацию о занятой и свободной памяти с помощью функции `GlobalMemoryStatus`, которая помещает информацию в структуру;
3. После вызова функции `GlobalMemoryStatus`, взять необходимые данные из структуры и поместим их в соответствующие элементы на форме;
4. Создать новую форму с таблицей, в которую будет выводиться информация обо всех процессах;
5. Сделать снимок памяти с помощью функции `CreateToolhelp32Snapshot`, первый параметр `TH32CS_SNAPPROCESS`, который указывает, что нам нужен снимок именно процессов;
6. С помощью функций `Process32First` и `Process32Next` заносится информация о процессе в структуру, а из структуры, соответственно, в таблицу;
7. Получить дескриптор процесса с помощью функции `OpenProcess` по его ID;
8. Используя функцию `GetProcessMemoryInfo` получить информацию об используемой памяти выбранного процесса, которую также занести в таблицу;
9. В конце работы закрыть все открытые дескрип

```

#include <windows.h>
#include <psapi.h>
#include <stdio.h>
#include <tchar.h>
#include <gdiplus.h>
#pragma comment(lib, "gdiplus.lib")

void PrintMemoryStatus() {
    MEMORYSTATUS memStatus;
    GlobalMemoryStatus(&memStatus);

    printf("Memory:\n");
    printf("Load: %ld%%\n", memStatus.dwMemoryLoad);
    printf("Total physical memory: %ld MB\n", memStatus.dwTotalPhys / (1024 * 1024));
    printf("Available physical memory: %ld MB\n", memStatus.dwAvailPhys / (1024 * 1024));
    printf("Total virtual memory: %ld MB\n", memStatus.dwTotalVirtual / (1024 * 1024));
    printf("Available virtual memory: %ld MB\n", memStatus.dwAvailVirtual / (1024 * 1024));
}

```

```

void DrawMemoryStatus(HDC hdc) {
    MEMORYSTATUS memStatus;
    GlobalMemoryStatus(&memStatus);

    int totalMemory = memStatus.dwTotalPhys / (1024 * 1024);
    int usedMemory = totalMemory - memStatus.dwAvailPhys / (1024 * 1024);

    // Used Memory Bar
    Gdiplus::Rect usedRect(20, 20, usedWidth, barHeight);
    graphics.Fill
    graphics.FillRectangle(&brush, usedRect);
}

```

```

// Free Memory Bar
Gdiplus::Rect freeRect(20 + usedWidth, 20, barWidth - usedWidth, barHeight);
brush.SetColor(Gdiplus::Color(255, 0, 255, 0));
graphics.FillRectangle(&brush, freeRect);
}

```

```

int main() {
    // Initialize GDI+
    Gdiplus::GdiplusStartupInput gdiplusStartupInput;
    ULONG_PTR gdiplusToken;
    Gdiplus::GdiplusStartup(&gdiplusToken, &gdiplusStartupInput, NULL);

    // Print memory status
    PrintMemoryStatus();

    // Print virtual memory map of the current process
    DWORD processID = GetCurrentProcessId();
    PrintVirtualMemoryMap(processID);

    // Shutdown GDI+
    Gdiplus::GdiplusShutdown(gdiplusToken);

    return 0;
}

```

Выбрать Консоль отладки Microsoft Visual Studio

```

Memory:
Load: 81%
Total physical memory: 16236 MB
Available physical memory: 3012 MB
Total virtual memory: 134217727 MB
Available virtual memory: 134213544 MB

```